

Documentación Técnica — Entrega 2

Proyecto: UTNotasApp

Grupo: 10 · DAM 2026

Integrantes: Andrada Santiago · Barbé Dante · Diez Nicolás · Soler Tomás

Fecha de entrega: 23 de junio de 2026

1. Autenticación y Autorización

1.1 Estrategia JWT

Los tokens se generan al hacer login (email/contraseña o Google OAuth) con una expiración de 7 días. Se transmiten de dos formas según el cliente:

- **Cookie HttpOnly (token):** para clientes web. El flag `httpOnly` previene acceso desde JavaScript, mitigando XSS.
- **Bearer token en header Authorization:** para clientes nativos (React Native / Expo Go), que no soportan cookies de la misma forma.

El middleware `auth.middleware.ts` verifica el token en ambos canales en el siguiente orden: primero el header `Authorization`, luego la cookie.

1.2 Roles de usuario

a. USER

b. ADMIN

Matriz de permisos

Módulo E2	Acción	Administrador	Usuario	Anónimo
Usuarios	Crear	sí	sí	sí
Usuarios	Consultar	sí	no	no
Usuarios	Consultar uno	sí	sí	no
Usuarios	Modificar	sí	sí	no
Usuarios	Eliminar	sí	sí	no
Materiales	Crear	sí	sí	no
Materiales	Consultar	sí	sí	sí
Materiales	Consultar uno	sí	sí	sí
Materiales	Modificar	sí	sí	no

Módulo E2	Acción	Administrador	Usuario	Anónimo
Materiales	Eliminar	sí	sí	no

Módulo E3	Acción	Administrador	Usuario	Anónimo
Carreras	Crear	sí	no	no
Carreras	Consultar	sí	sí	sí
Carreras	Consultar uno	sí	sí	sí
Carreras	Modificar	sí	no	no
Carreras	Eliminar	sí	no	no
Materias	Crear	sí	no	no
Materias	Consultar	sí	sí	sí
Materias	Consultar uno	sí	sí	sí
Materias	Modificar	sí	no	no
Materias	Eliminar	sí	no	no
Calificación	Crear	sí	sí	no
Calificación	Consultar	sí	sí	sí
Calificación	Consultar uno	sí	sí	sí
Calificación	Modificar	sí	sí	no
Calificación	Eliminar	sí	sí	no

1.3 Autorización basada en atributos (ABAC)

Se implementó control de acceso por atributos para recursos propios:

Usuarios

PATCH /api/users/:id y DELETE /api/users/:id verifican que el id del JWT coincida con el :id del parámetro de ruta y que el usuario tenga rol ADMIN.

Materiales

PATCH /api/materials/:id y DELETE /api/materials/:id verifican que el userId del material en la base de datos coincida con el id del JWT y que el rol sea ADMIN.

Si la verificación falla se responde con 403 Forbidden.

2. API / Backend

2.1 Nuevos endpoints en E2

PATCH /api/users/:id

Actualiza los datos del perfil del usuario.

Campo	Tipo	Descripción
name	string	Nombre (opcional)
surname	string	Apellido (opcional)
username	string	Nombre de usuario (opcional)
password	string	Nueva contraseña (opcional)
currentPassword	string	Requerida para confirmar el cambio

- Requiere token JWT válido.
 - Responde con el usuario actualizado (sin contraseña).
-

DELETE /api/users/:id

Elimina la cuenta del usuario.

Campo	Tipo	Descripción
password	string	Requerida para confirmar

- Requiere token JWT válido.
 - Verifica la contraseña antes de eliminar.
 - Al eliminar, invalida la cookie del token.
-

2.2 Endpoints de E1 (sin cambios de interfaz)

Método	Ruta	Descripción
POST	/api/auth/login	Login email/contraseña
POST	/api/auth/logout	Cierre de sesión
GET	/api/auth/me	Usuario autenticado
POST	/api/users/register	Registro
GET	/api/users/:id	Perfil público
GET	/api/materials	Listar materiales (con filtros y paginación)

Método	Ruta	Descripción
GET	/api/materials/:id	Detalle de material
POST	/api/materials	Subir material
PATCH	/api/materials/:id	Editar material (+ ownership check E2)
DELETE	/api/materials/:id	Eliminar material (+ ownership check E2)

2.3 Manejo de errores de red y estrategia de retry

- **Cliente HTTP centralizado:** `apiFetch` envuelve `fetch`, inyecta el `Bearer token`, y ante una respuesta `!res.ok` o un JSON inválido lanza un `ApiError(status, message)`, unificando el manejo de errores en un solo punto.
- **Mapeo a errores de dominio:** cada servicio captura el `ApiError` y lo convierte en un `MaterialError` con un `code` semántico (`NOT_FOUND`, `MATERIAL_NOT_OWNED`, `UNKNOWN`) y un mensaje en español apto para mostrar al usuario, separando el error técnico del mensaje de UI.
- **Sin retry en escrituras:** las mutaciones (`useCreateMaterial`, `useUpdateMaterial`, `useDeleteMaterial`) usan `retry: false` explícito, registran el fallo (`[CRITICAL_UI_ERROR]`) y exponen el error a la pantalla, evitando reintentos automáticos en operaciones no idempotentes.
- **Reintento manual en lecturas:** las queries (`useGetMaterials`, `useGetMaterial`) se apoyan en React Query con `staleTime` para cachear y exponen `refetch/isEmpty`, permitiendo que el usuario dispare el reintento de forma deliberada.
- **Feedback de error con reintento:** el componente reusable `ErrorState` muestra el estado de fallo (con rol de accesibilidad `alert`) y un botón de "reintentar" opcional cableado a la acción `onRetry` (típicamente el `refetch` de la query).

3. Decisiones de Refactoring

3.1 useMaterial hook — isOwner

El hook `useMaterial.ts` ahora expone el booleano `isOwner`, calculado comparando `material.userId` con el id del usuario en sesión. `MaterialDetailScreen` usa este valor para mostrar u ocultar los botones de editar y eliminar.

4. Seguridad

Medida	Implementación
Hash de contraseñas	bcryptjs
Autenticación	JSON Web Token
Validación de entrada	Zod en todos los endpoints
Field whitelisting	Solo se permiten campos explícitamente listados en updates
Limitar acceso a materiales	Middleware ABAC en recursos de usuario y material
Protección timing attack	bcrypt.compare() siempre ejecutado en login

5. Deuda técnica documentada

Item	Estado	Plan E3
Foto de perfil	Pendiente	Upload a almacenamiento de archivos (S3 o similar)
reCAPTCHA / hCaptcha	Stub preparado	Activar con CAPTCHA_ENABLED=true
panel de ADMIN	Pendiente	Crear el panel de administrador para eliminar materiales inapropiados
